

课程讲义

大模型训练并行：从集合通信到 **3D** 并行

Stanford CS336 Spring 2025 · Lecture 7 · Parallelism 1



频道：Stanford Online

发布日期：2025 年 5 月 6 日

视频时长：1 小时 24 分 42 秒

视频链接：<https://www.youtube.com/watch?v=l1RJcDjzK8M>

整理说明：依据完整字幕和原始课件重构；所有内存与通信公式都标明口径。目标是能根据显存、带宽、批量和拓扑组合数据、张量、流水线与序列并行。

整理日期：2026 年 7 月 29 日

目录

1	为什么单 GPU 缩放会停止	3
1.1	算力上限与显存上限	3
1.2	本章小结	5
2	集合通信与网络拓扑	6
2.1	从点对点到 collectives	6
2.2	本章小结	8
3	朴素数据并行及其显存账本	9
3.1	每卡不同样本, 梯度全局同步	9
3.2	本章小结	11
4	ZeRO-1 与 ZeRO-2	11
4.1	先分最贵的优化器状态	11
4.2	继续分片梯度	13
4.3	本章小结	14
5	ZeRO-3 / FSDP: 参数也按需物化	14
5.1	分片一切	14
5.2	本章小结	18
6	数据并行的计算缩放边界	18
6.1	全局批量不能无限增大	18
6.2	本章小结	19
7	流水线并行: 沿深度切分	20
7.1	从按层串行到 microbatch 流水	20
7.2	本章小结	22
8	张量并行、激活显存与序列并行	23
8.1	沿宽度切矩阵	23
8.2	本章小结	26
9	组合并行与工程决策	27
9.1	没有单一最优原语	27
9.2	本章小结	29

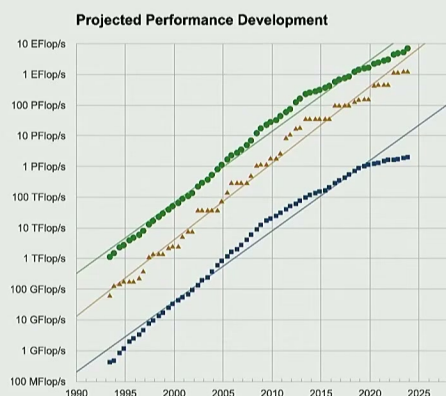
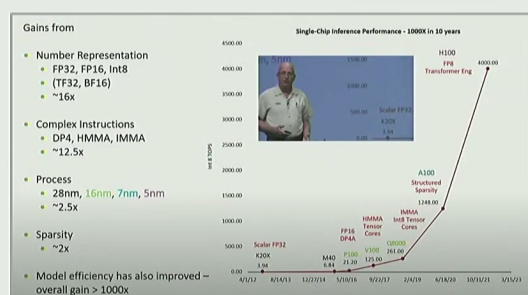
10 并行方案设计工作坊	30
10.1 先建立完整资源账本	30
10.2 从硬件拓扑反推并行维度	30
10.3 选择 FSDP 粒度	30
10.4 流水线调度与 batch	31
10.5 张量、序列与上下文轴	31
10.6 用实验关闭决策	31
10.7 复习检查	32
10.8 本章小结	32

1 为什么单 GPU 缩放会停止

1.1 算力上限与显存上限

大模型训练遇到两个独立边界：一张卡的 FLOPs 无法在可接受时间内完成总工作量，一张卡的 HBM 也装不下参数、梯度、优化器状态和激活。并行系统的目标因而有两条：让可训练模型大小随设备数近线性增长，让有效计算吞吐也随设备数近线性增长。

Limits to GPU-based scaling – compute



There are limits to single-GPU scaling.

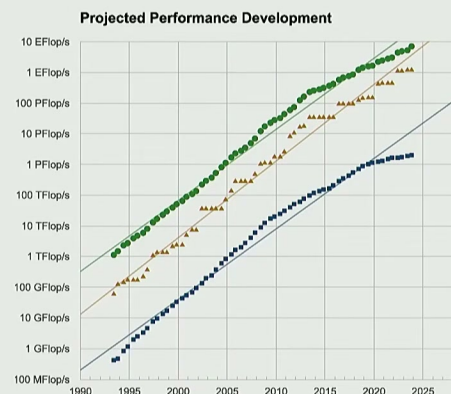
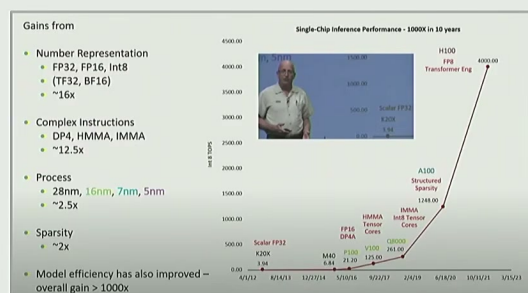
The world's fastest supercomputers have *exaflops* of compute

Stanford

图 1：单设备计算扩展存在物理上限，超级计算机以 exaFLOP 计。

视频证据时间窗：00:01:25–00:02:05。

Limits to GPU-based scaling – compute



There are limits to single-GPU scaling.
The world's fastest supercomputers have *exaflops* of compute

Stanford

图 2: 模型参数量增长使单卡显存很快失效。

视频证据时间窗: 00:02:10–00:02:50。

若总训练 FLOPs 为 F , 设备数为 p , 单设备有效吞吐为 R , 并行效率为 η_p , 则

$$T_p = \frac{F}{pR\eta_p}.$$

- F : 固定训练任务的总工作量;
- pR : 理想聚合吞吐;
- η_p : 通信、同步、空泡和负载不均造成的折损。

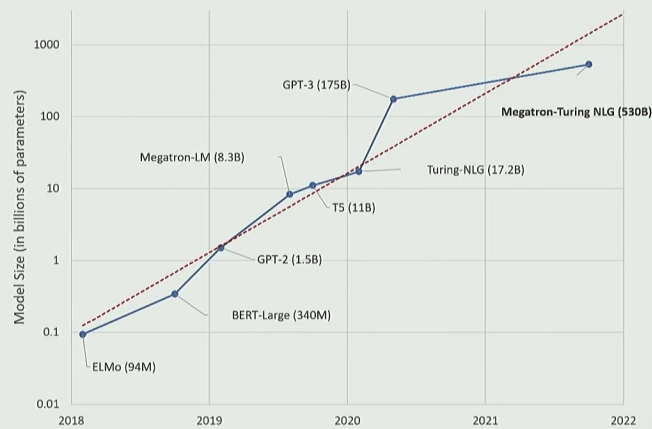
若每卡显存为 M_{gpu} , 理想可用总显存为

$$M_{\text{total}} = pM_{\text{gpu}}.$$

- 只有状态真正分片时, 总显存才可近线性利用;
- 复制模型的数据并行不会扩大单模型可容纳参数量;
- 激活峰值和临时 buffer 还会占用预算。

Limits to GPU-based scaling - memory

Models are getting really big..



A single GPU can't fit most of these large models!

Stanford

图 3: 多 GPU、多机系统同时包含高速节点内和较慢节点间链路。

视频证据时间窗: 00:02:55–00:03:35。

并行优化是约束满足，不是单指标最大化

更多设备不一定更快。方案必须同时让模型装得下、每卡有足够工作、通信可隐藏、全局批量仍处于有效统计区间。任一约束失败，都可能得到“花更多 GPU 却更慢”的结果。

1.2 本章小结

并行训练不是可选的加速技巧，而是超大模型的容量前提。计算缩放和内存缩放必须分开核算，再由拓扑与批量把两者统一。

2 集合通信与网络拓扑

2.1 从点对点到 collectives

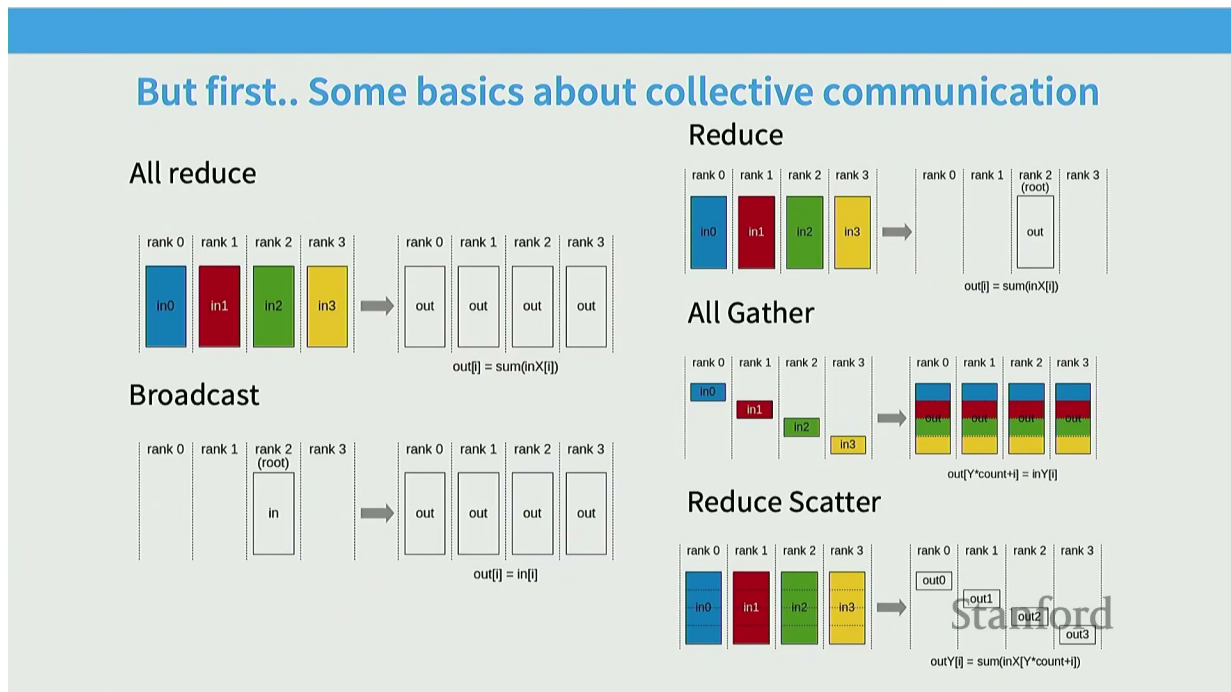


图 4: broadcast、reduce、all-gather、reduce-scatter 与 all-reduce。

视频证据时间窗：00:04:40-00:05:20。

设有 p 个 rank，每个 rank 持有张量 x_r 。all-reduce 求

$$y_r = \bigoplus_{j=0}^{p-1} x_j, \quad r = 0, \dots, p-1.$$

- $\oplus$ ：可结合且可交换的归约，例如 sum、min、max；
- y_r ：每个 rank 最终都得到相同结果；
- 梯度同步通常选择求和或平均。

带宽受限的大消息 ring all-reduce，每 rank 的近似通信量是

$$V_{AR} \approx 2 \frac{p-1}{p} S.$$

- S ：输入张量字节数；
- 前一项来自 reduce-scatter，后一项来自 all-gather；
- p 很大时每 rank 约传输 $2S$ ，而不是 pS 。

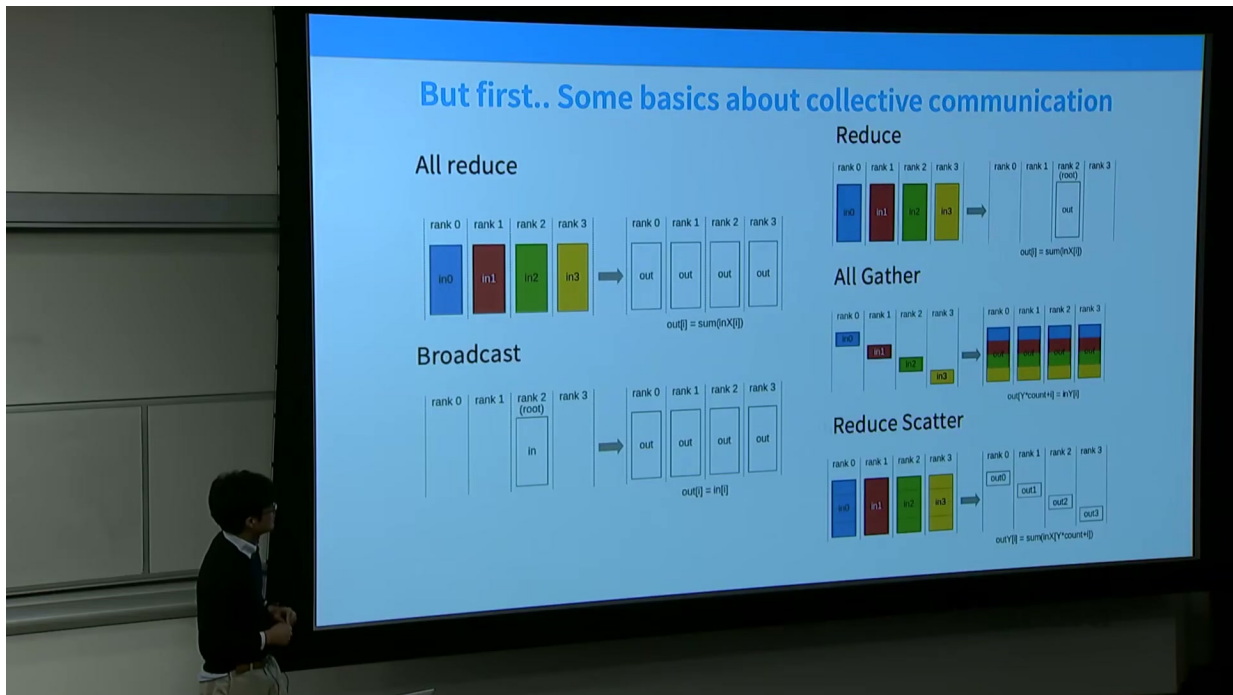


图 5: all-reduce 可拆为 reduce-scatter 再 all-gather。

视频证据时间窗：00:06:25–00:07:05。

拆分原语是后续 ZeRO 的钥匙

all-reduce 不必视作不可分黑盒。若训练只需要每个 rank 获得梯度的一片，可在 reduce-scatter 后直接更新；只有随后需要完整参数时再 all-gather。这种“恰好在需要时聚合”的思想贯穿 ZeRO/FSDP。

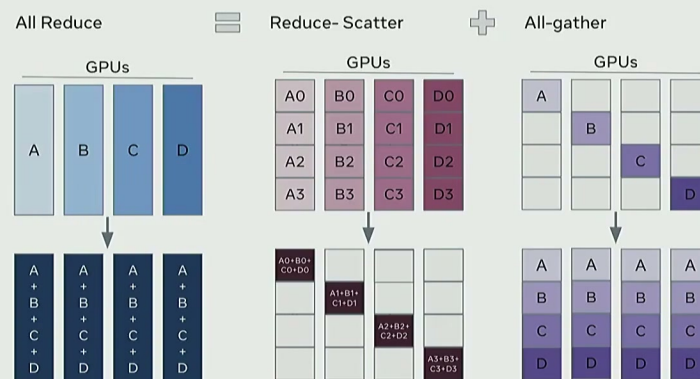
通信时间常用 α - β 模型：

$$T_{\text{comm}} \approx n_{\text{msg}}\alpha + \frac{V}{B}.$$

- α ：每条消息启动和传播的延迟；
- n_{msg} ：算法步数或消息数；
- V ：总传输字节；
- B ：有效链路带宽。

Important detail – all reduce vs reduce-scatter-gather.

Reduce can be implemented as two steps: reduce-scatter and all-gather



Importantly, in the bandwidth-limited regime, this is the best you can do

图 6: TPU 环面与 GPU NVLink/NVSwitch 的通信拓扑不同。

视频证据时间窗: 00:08:05–00:08:45。

相同通信量不代表相同时间

节点内 NVLink/NVSwitch 与节点间网络的带宽和延迟差异很大。并行维度要映射到物理拓扑：高频张量并行放在高速域，低频流水线通信可跨较慢链路。

2.2 本章小结

集合通信给出算法词汇，拓扑决定实际代价。reduce-scatter 和 all-gather 的分解让“复制”与“分片”可以按训练阶段重新组合。

3 朴素数据并行及其显存账本

3.1 每卡不同样本，梯度全局同步

Part 2 – Standard LLM parallelization primitives

How do we parallelize LLMs? 3 important ideas

- **Data parallelism**
 - Naïve data parallel
 - ZeRO levels 1-3
- **Model parallelism**
 - Pipeline parallel
 - Tensor parallel
- **Activation parallelism**
 - Sequence parallel

Stanford

图 7: 数据、模型和激活并行的标准分类。

视频证据时间窗: 00:12:30–00:13:10。

Naïve data parallelism

Starting point – imagine we are doing naïve SGD

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^B \nabla f(x_i)$$

Naïve parallelism: split the elements of B sized batch across M machines. Exchange gradients to synchronize

How does this do?

Compute scaling – each GPU gets B/M examples.

Communication overhead – transmits 2x # params every batch. OK if batches are big

Memory scaling – none. Every GPU needs # params at least

Stanford

图 8: 每卡处理 B/p 样本，反向后同步梯度。

视频证据时间窗: 00:14:05–00:14:45。

若第 r 个 rank 的局部批量为 $\mathcal{B}_r$, 全局平均梯度是

$$g = \frac{1}{p} \sum_{r=0}^{p-1} \frac{1}{|\mathcal{B}_r|} \sum_{x \in \mathcal{B}_r} \nabla_{\theta} \ell(x; \theta).$$

- $\mathcal{B}_r$: 第 r 个 rank 的样本子集;
- p : 数据并行度;
- 同步后所有 rank 用相同 g 更新同一份参数副本。

课程以混合精度 Adam 为例, 把每参数状态近似记为 16 字节:

$$M_{\text{DDP}} \approx 16P.$$

- P : 参数个数;
- 可包括 BF16 参数 2 字节、BF16 梯度 2 字节、FP32 master weight 4 字节、两个 FP32 Adam moment 共 8 字节;
- 不同实现和精度会改变常数, 但复制随 p 不下降。

Naïve data parallelism

Starting point – imagine we are doing naïve SGD

$$\theta_{t+1} = \theta_t - \eta \sum_{i=1}^B \nabla f(x_i)$$

Naïve parallelism: split the elements of B sized batch across M machines. Exchange gradients to synchronize

How does this do?

Compute scaling – each GPU gets B/M examples.

Communication overhead – transmits $2 \times \#$ params every batch. OK if batches are big

Memory scaling – none. Every GPU needs $\#$ params at least

Stanford

图 9: 混合精度 Adam 的参数、梯度与优化器状态账本。

视频证据时间窗: 00:15:25–00:16:05。

先做每参数账本

讨论“模型能否装下”前, 明确参数、梯度、master weight、moment、激活、临时 buffer 的数据类型和生命周期。只用参数文件大小估算训练显存, 通常会低估数倍。

朴素 DDP 的每步梯度通信量近似

$$V_{\text{DDP}} \approx 2 \frac{p-1}{p} s_g P.$$

- s_g : 每个梯度元素字节数;
- ring all-reduce 的系数来自两阶段通信;
- 计算量随局部批量缩小, 通信量却主要由 P 决定。

3.2 本章小结

DDP 易用且计算缩放直接, 但每卡完整复制模型状态。它的容量不随设备数增长, 并在局部批量变小时逐渐被固定的参数级通信支配。

4 ZeRO-1 与 ZeRO-2

4.1 先分最贵的优化器状态

What's wrong with naïve data parallelism? - Memory

Our memory situation is actually terrible.

Depending on our precision..

We need 5 copies of weights and 16 bytes per param!

- 2 bytes for FP/BF 16 model parameters
- 2 bytes for FP/BF 16 gradients
- 4 bytes for FP32 master weights (the thing you accumulate into in SGD)
- 4 (or 2) bytes for FP32/BF16 Adam first moment estimates
- 4 (or 2) bytes for FP32/BF16 Adam second moment estimates

“Optimizer state”

Stanford

图 10: ZeRO 逐级分片优化器状态、梯度和参数。

视频证据时间窗: 00:17:25–00:18:05。

ZeRO-1 只分片优化器状态, 每卡保留完整参数和梯度。若参数与梯度合计每参数 a 字节, 优化器状态为 b 字节, 则

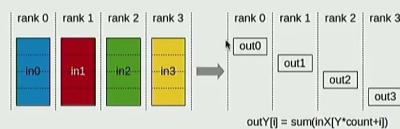
$$M_{Z1} = aP + \frac{bP}{p}.$$

- a : 未分片的参数与梯度字节;
- b : 可分片的 master weight 与 moment 字节;
- 设备数增加时, 最昂贵的 bP 项线性下降。

ZeRO stage 1. how it works

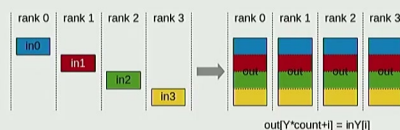
Step 1. Everyone computes a full gradient on their subset of the batch

Step 2. ReduceScatter the gradients – incur #params communication cost



Step 3. Each machine updates their param using their gradient + state.

Step 4. All Gather the parameters – incur #params communication cost



Stanford

图 11: ZeRO-1: 每个 rank 只更新自己负责的参数片。

视频证据时间窗: 00:21:40–00:22:20。

其通信流程为梯度 reduce-scatter、局部更新、参数 all-gather。带宽主导时总量仍近似

$$V_{Z1} \approx 2 \frac{p-1}{p} sP.$$

- 第一半是 reduce-scatter 梯度;
- 第二半是 all-gather 更新后的参数;
- 与 DDP 的 all-reduce 相当, 因此 ZeRO-1 的显存收益近似“免费”。

Comparing ZeRO stage 1 and naïve data parallel

	Naïve DDP	ZeRO stage 1
Communication primitive	One all-reduce (gradients)	One reduce scatter (send gradients) + all gather (collect params)
Communication cost	$2 * \# \text{ params}$	$2 * \# \text{ params}$
Memory	$(4+K) * \# \text{ params}$	$(4+K/N_{\text{gpu}}) * \# \text{ params}$

ZeRO stage 1 is *free* (in the bandwidth limited regime) memory wins

Stanford

图 12: ZeRO-1 和 DDP 的通信量相近，显存占用更低。

视频证据时间窗：00:25:25–00:26:05。

“免费”有明确前提

这里指大消息、带宽主导且实现高效时的总字节量。真实系统还会受 collective 启动、bucket、流同步和参数 materialization 影响；不能把通信公式直接当作零开销保证。

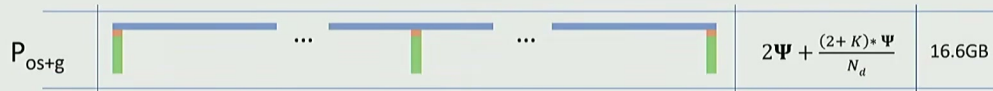
4.2 继续分片梯度

ZeRO-2 让每个 rank 只保留自己的梯度片：

$$M_{Z2} = m_p P + \frac{(m_g + m_o)P}{p}.$$

- m_p ：完整参数每元素字节；
- m_g ：梯度字节；
- m_o ：优化器状态字节；
- 反向一得到某层梯度就 reduce-scatter，并可释放完整梯度。

ZeRO stage 2. the simple extension to gradient sharding



Emboldened by our success, let's shard even more stuff

High level idea

- Also keep the gradients (pink slices) sharded across the machines.
- Use the same (rough) tricks as stage 1.

Complexity – we can never instantiate a full gradient vector, but each worker must compute a full gradient (since we're data parallel)

Stanford

图 13: ZeRO-2 在 ZeRO-1 基础上继续分片梯度。

视频证据时间窗: 00:27:10–00:27:50。

生命周期比静态公式重要

若先生成完整梯度再统一分片，峰值仍可能很高。有效 ZeRO-2 要把通信挂到反向图上：梯度一旦就绪就归约、分片并释放。内存优化本质上是时间轴优化。

4.3 本章小结

ZeRO-1 分片优化器，ZeRO-2 再分片梯度；它们在不改变模型数学的前提下重排状态所有权。收益来自把复制状态变成按 rank 所有，而不是减少状态本身。

5 ZeRO-3 / FSDP：参数也按需物化

5.1 分片一切

ZeRO-3 (FSDP) 连参数也分片。静态模型状态近似

$$M_{Z3} \approx \frac{(m_p + m_g + m_o)P}{p}.$$

- 参数、梯度和优化器状态都按 rank 分片；
- 每层计算前 all-gather 所需参数；
- 计算后释放非本地参数，反向梯度用 reduce-scatter。

ZeRO stage 3 (aka FSDP) shard everything



We've gotten almost everything for free so far.. lets try to solve *all* our memory issues

High level idea

- Shard everything – incl parameters!
- Use the same 'incremental communication / computation' ideas
- Send and request parameters on demand while stepping through the compute graph.

Is it possible to do this with low overhead?

Stanford

图 14: FSDP 按模块请求完整参数，完成后重新分片。

视频证据时间窗：00:30:10–00:30:50。

单层参数量为 P_ℓ 时，前后向各自需要一次参数物化，近似通信量为

$$V_\ell \approx 3 \frac{p-1}{p} s P_\ell.$$

- 两次 all-gather：前向一次、反向前一次；
- 一次 reduce-scatter：反向梯度；
- 系数约 3，通常是 DDP 参数通信的 1.5 倍。

Actual picture of how FDSP / ZeRO stage 3 works

Let's walk through a FSDP example to see some important ideas

Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

Overlapping communication and computation ($W_1 W_0 + W_2 W_0$) $x = y$

- The all-gathers happen all at once while forward happens, masking the comm cost.

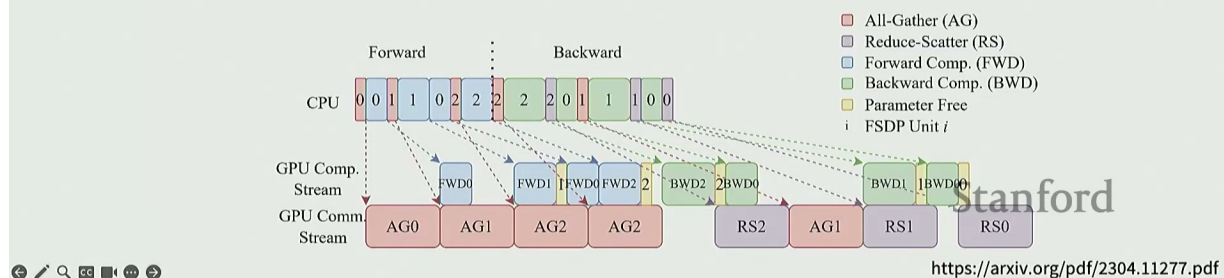


图 15: 预取下一层参数并与当前层计算重叠。

视频证据时间窗: 00:34:30-00:35:10。

重叠后的关键路径近似

$$T_{\text{layer}} \approx \max(T_{\text{compute}}, T_{\text{comm}}) + T_{\text{exposed}}$$

- 前两项可通过异步流重叠;
- T_{exposed} : 无法预取、尾部或同步造成的暴露通信;
- bucket 和 wrap 粒度决定是否有足够计算遮蔽下一次 all-gather。

Actual picture of how FSDP / ZeRO stage 3 works

Let's walk through a FSDP example to see some important ideas

Incremental computation / communication

- Parameters / gradients are requested / sent and then immediately freed

Overlapping communication and computation ($W_1 W_0 + W_2 W_0$) $x = y$

- The all-gathers happen all at once while forward happens, masking the comm cost.

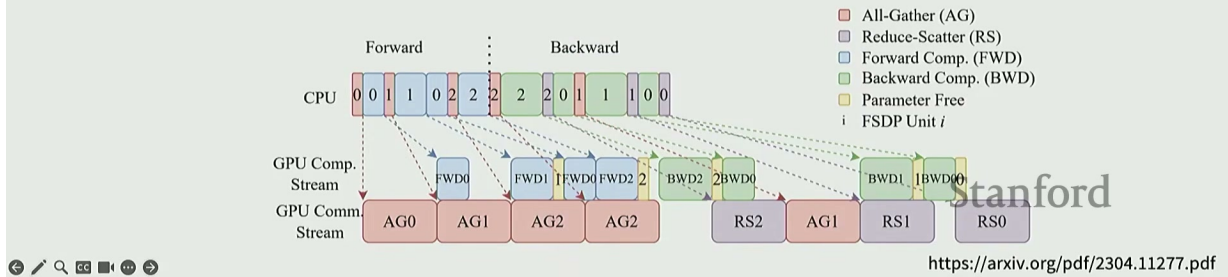


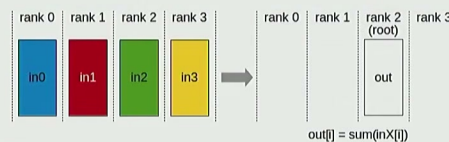
图 16: ZeRO-1/2 约 $2P$ 通信, ZeRO-3 约 $3P$ 通信。

视频证据时间窗: 00:40:25–00:41:05。

ZeRO stage 2. how it works

Step 1. Everyone incrementally goes backward on the computation graph

Step 1a. After computing a layer's gradients, immediately reduce to send this to the right worker



Step 1b. Once gradients are not needed in the backward graph, immediately free it.

Step 2. Each machine updates their param using their gradient + state.

Step 3. All Gather the parameters.

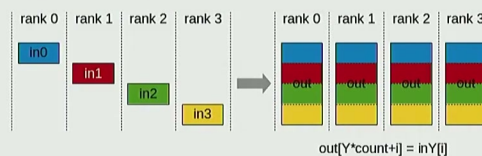


图 17: 8 张 80GB A100 的容量示例: 基线约 6.66B, ZeRO-3 约 53.33B。

视频证据时间窗: 00:41:25–00:42:05。

FSDP 不减少激活显存

ZeRO-3 解决模型状态复制, 却不自动分片残差流、注意力中间量等激活。长序列、大 microbatch 时, 激活仍可主导峰值; 这正是后续张量、序列和检查点技术的动机。

wrap 粒度的取舍

模块太大：一次物化参数峰值高，通信启动少；模块太小：峰值低，却产生许多小 collective 和调度开销。最佳粒度要让单块计算足以覆盖下一块通信，同时满足显存上限。

5.2 本章小结

FSDP 用更多参数通信换近线性状态显存缩放。按需物化、立即释放和通信计算重叠是三个不可分割的实现条件。

6 数据并行的计算缩放边界**6.1** 全局批量不能无限增大

数据并行要求设备数不超过可分配 microbatch 数。若每卡局部批量至少为 $b_{\min}$ ，则

$$p \leq \frac{B_{\text{global}}}{b_{\min}}.$$

- B_{global} ：全局批量；
- $b_{\min}$ ：保持 kernel 利用率和统计效率所需的最小局部批量；
- 超过此界要增加全局批量或换模型并行。

Issues remain with data parallel – compute scaling

With data parallel, **#machines < batch size** (and near this, comm overhead is high)

And there's diminishing returns to batch sizes

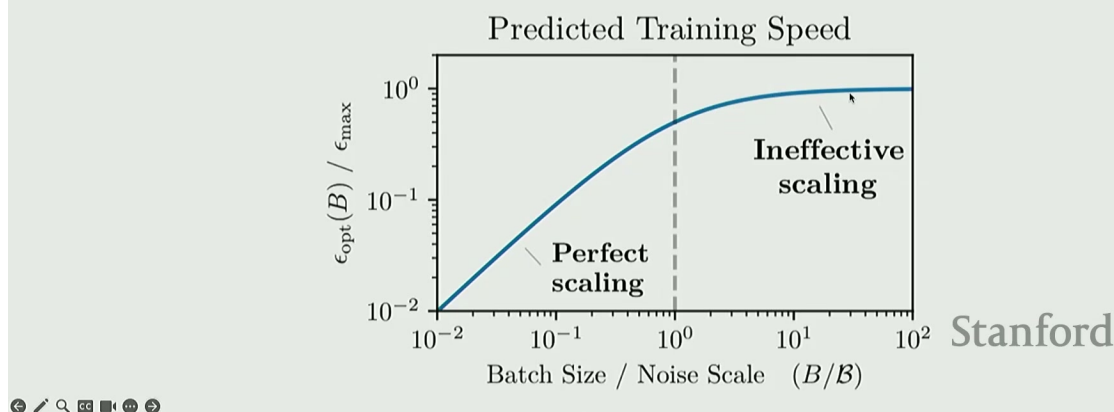


图 18: 批量增益出现递减，数据并行不能无限横向扩展。

视频证据时间窗：00:43:25–00:44:05。

若梯度累积 a 次、每卡 microbatch 为 b ，则

$$B_{\text{global}} = pab.$$

- p : 数据并行度;
- a : 梯度累积步数;
- b : 单卡每次前后向样本数;
- 增大 a 不增加同步频率, 却延长一次优化步。

Beyond data parallel – model parallelism

Scaling up in memory (without changing batch size) with model parallelism

What model parallelism is..

- It splits up the parameters across GPUs (like zero3)..
- But communicate activations (while zero3 sends params).

We cover two different types of parallelism

1. Pipeline parallel
2. Tensor parallel (+ Sequence parallel)

These correspond to two different ways of cutting up the model.

Stanford

图 19: 模型并行切参数或激活, 使单样本可跨多卡计算。

视频证据时间窗: 00:45:25–00:46:05。

不要用更大批量掩盖低效率

增大 B_{global} 可以让更多数据并行 rank 有活干, 却可能改变优化动力学和所需训练 token。系统吞吐提高不等于达到同等损失的总时间下降, 必须同时看 time-to-quality。

6.2 本章小结

DDP/FSDP 的数据轴最终受全局批量和局部 kernel 效率约束。设备继续增加时, 需要沿模型深度或宽度切分。

7 流水线并行：沿深度切分

7.1 从按层串行到 microbatch 流水

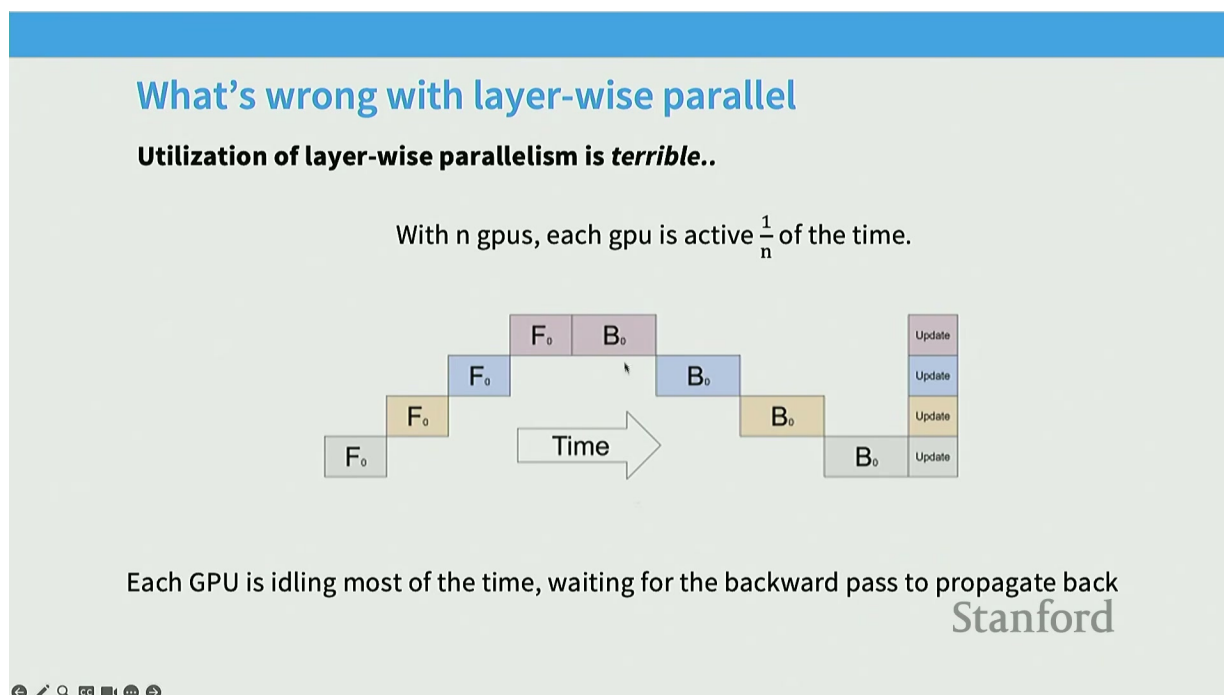


图 20: 朴素按层切分使每张 GPU 只在 $1/p$ 时间内工作。

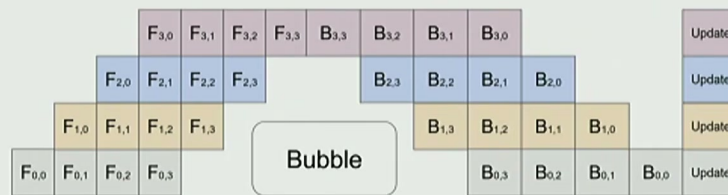
视频证据时间窗：00:47:10-00:47:50。

若流水线有 p 个 stage、一次迭代切成 m 个 microbatch，简单 GPipe 调度的气泡比例近似

$$f_{\text{bubble}} \approx \frac{p-1}{m+p-1}.$$

- $p-1$: 流水线填充与排空的空闲阶梯;
- m : 有效 microbatch 数;
- $m \gg p$ 时气泡比例下降，但激活排队和批量需求增加。

A solution: pipeline parallel



Solution: Pipeline-parallel.

Process 'micro-batches' (in this case, 4).
Send off the first microbatch and start computing the second.

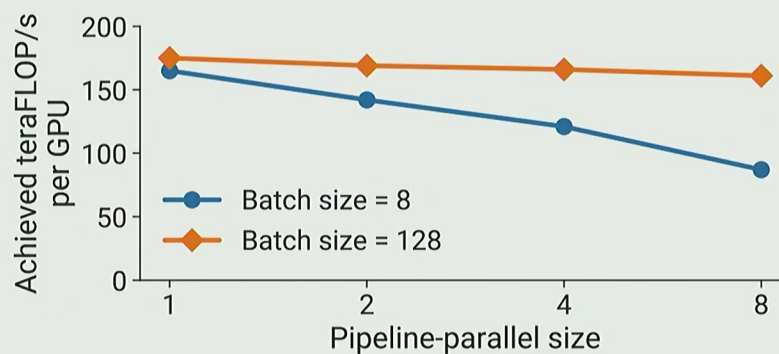
The ratio of bubble time to useful compute is $\dots \frac{n_{stages}-1}{n_{micro}}$ so we need a big batch size!



图 21: microbatch 让不同 stage 同时处理不同样本。

视频证据时间窗: 00:49:05-00:49:45。

Pipeline performance is highly dependent on batch size



Batch sizes are key to hiding the bubble – otherwise pipeline rapidly degrades perf

图 22: 小批量无法隐藏气泡, stage 数增加时吞吐下降。

视频证据时间窗: 00:50:25-00:51:05。

流水线切分还要负载均衡

公式假设各 stage 用时相近。若某 stage 包含 embedding、输出头或更重的层, 所有其他 stage 都要等待。分区目标应是时间和显存平衡, 而非简单地平均层数。

点对点传输一个激活张量的字节量为

$$V_{\text{act}} = bshd_{\text{bytes}}.$$

- b : microbatch 大小;
- s : 序列长度;
- h : 隐藏维度;
- d_{bytes} : 每元素字节数;
- 相比参数级 collective, 流水线只在 stage 边界传激活。

'Zero bubble' pipelining

Split up backwards into two parts

1. Backpropagating activations (z, x)
2. Computing weight gradients (W)

The second part can be done whenever

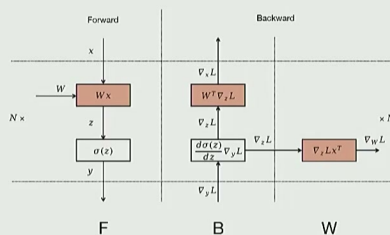


Figure 1: Computation Graph for MLP.



Figure 2: 1F1B pipeline schedule.

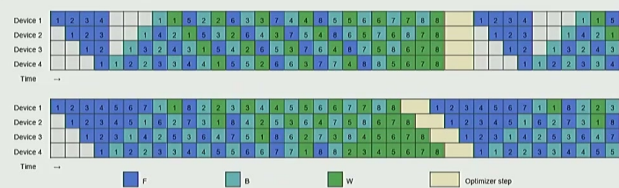


Figure 3: Handcrafted pipeline schedules, top: ZB-H1; bottom: ZB-H2

Stanford

图 23: 将反向拆成输入梯度与权重梯度, 可更灵活填充空档。

视频证据时间窗: 00:52:05-00:52:45。

零气泡的核心

反向传播包含向前一 stage 传递的激活梯度和本地权重梯度。前者在关键依赖链上, 后者可延后调度。拆分两类工作, 就能把权重梯度塞进原本的空泡, 但会增加调度和缓冲复杂度。

7.2 本章小结

流水线并行以低频激活通信换模型容量, 适合较慢的节点间链路。它的代价是气泡、microbatch 依赖和 stage 负载均衡。

8 张量并行、激活显存与序列并行

8.1 沿宽度切矩阵

矩阵乘 $Y = XA$ 可把 A 按列分为 $[A_1, \dots, A_p]$:

$$Y = [XA_1, \dots, XA_p].$$

- 每个 rank 计算一组输出特征;
- 列并行产生分片输出;
- 后续行并行可局部计算再 all-reduce 部分和。

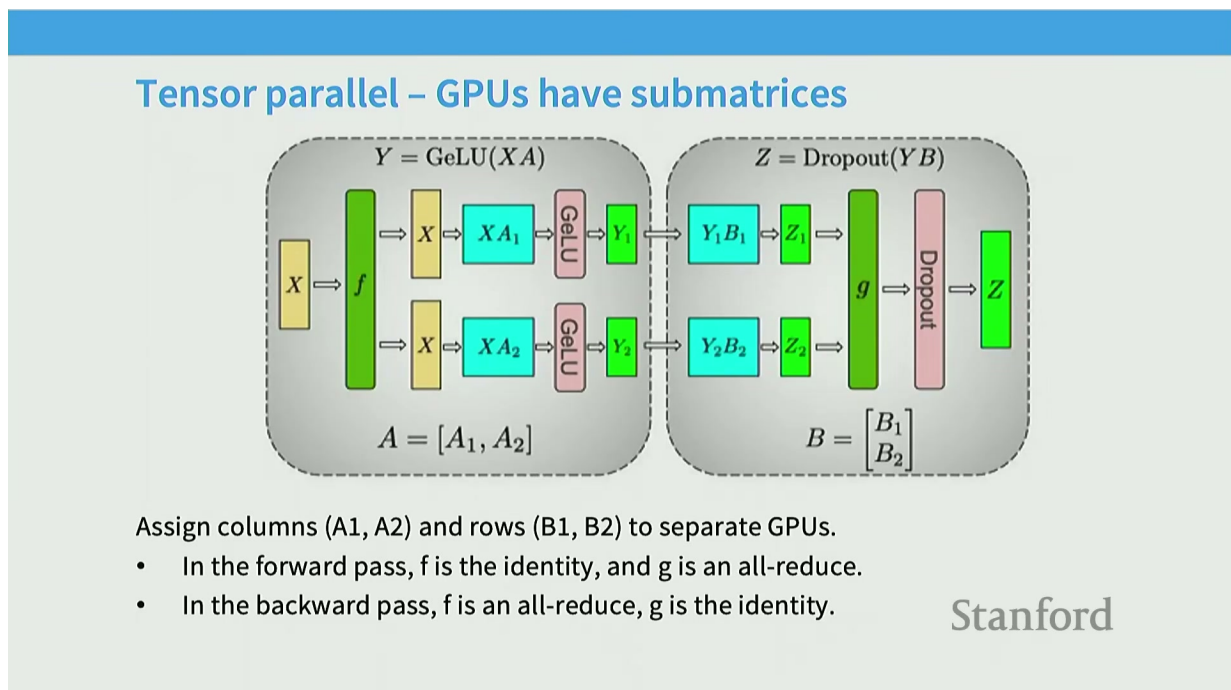


图 24: Transformer MLP 以列并行和行并行组合切分矩阵。

视频证据时间窗: 00:56:40–00:57:20。

若每层张量并行需要两次 all-reduce、激活张量为 A 字节，通信近似

$$V_{TP} \approx 2 \cdot 2^{\frac{p-1}{p}} A.$$

- 外层系数 2: 一个 Transformer block 中常见两处同步;
- ring all-reduce 系数来自 reduce-scatter 与 all-gather;
- 张量并行通信频繁，因此通常放在高速节点内。

A final complexity – memory is dynamic!

Memory isn't just the static bits, but also activations! This can be big

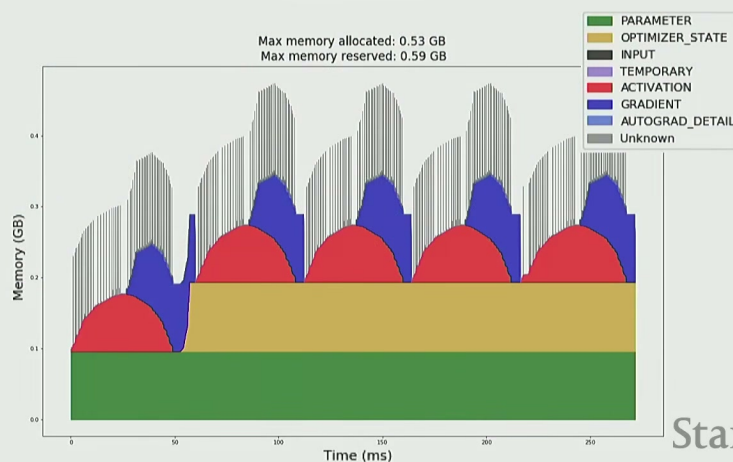


图 25: 训练峰值同时包含静态模型状态与随时间变化的激活。

视频证据时间窗: 01:03:10–01:03:50。

课程给出的“保存全部中间量”每层激活估算为

$$M_{\text{act}} = sbh \left(34 + 5 \frac{as}{h} \right).$$

- s : 序列长度, b : microbatch, h : 隐藏维度;
- a : 注意力头数;
- 二次项来自注意力概率等 s^2 中间量, 可由 FlashAttention/重计算去掉。

What's the activation memory per layer?

Starting point: activation memory needed if storing everything

$$\text{Activations memory per layer} = sbh \left(34 + 5 \frac{as}{h} \right).$$

- The $5 \frac{as}{h}$ terms come from the quadratic attention terms incl dropout
- As with flash attention, we can drop this term via recomputation

a	number of attention heads	p	pipeline parallel size
b	microbatch size	s	sequence length
h	hidden dimension size	t	tensor parallel size
L	number of transformer layers	v	vocabulary size

图 26: 激活显存含线性项与注意力二次项。

视频证据时间窗: 01:06:05–01:06:45。

参数装得下不等于训练装得下

FSDP 的状态显存可随设备数下降, 但激活随 batch、序列长度和层数增长。长上下文训练必须把激活账本单独列出, 并结合 FlashAttention、activation checkpointing、张量与序列并行。

张量并行后, 每层激活近似变为

$$M_{\text{act,TP}} = sbh \left(10 + \frac{24}{p} + 5 \frac{as}{hp} \right).$$

- 许多矩阵乘中间量被 p 分片;
- layer norm、dropout 等点式算子的若干项仍复制;
- 因此张量并行并不能让全部激活严格按 $1/p$ 缩放。

序列并行进一步沿 s 切分这些复制的点式激活:

$$M_{\text{act,SP}} \propto \frac{sbh}{p}.$$

- layer norm 与 dropout 在序列维上可独立计算;
- 前向在张量分片与序列分片之间用 all-gather/reduce-scatter 转换;
- 反向执行相反的 collective。

Activation under tensor parallel

$$\text{Activations memory per layer} = sbh \left(10 + \frac{24}{t} + 5 \frac{as}{ht} \right)$$

Tensor parallel splits out the matrix multiplies in attention + MLP

The remaining **10** term is for the LayerNorm (4sbh), Dropout (2sbh), and inputs to the attention and MLP (4sbh). These terms alone will continue to grow with size

a	number of attention heads	p	pipeline parallel size
b	microbatch size	s	sequence length
h	hidden dimension size	t	tensor parallel size
L	number of transformer layers	v	vocabulary size

图 27: 在点式算子处沿序列切分, 使激活显存真正线性缩放。

视频证据时间窗: 01:07:25–01:08:05。

为什么张量与序列并行成对出现

张量并行擅长切矩阵乘, 却留下复制的 layer norm/dropout 输入; 序列并行正好切这些点式激活。二者共享同一设备组, 并用 all-gather 与 reduce-scatter 在布局之间转换。

8.2 本章小结

张量并行沿宽度分摊参数计算, 序列并行补上点式激活的显存缩放。它们通信频繁但没有流水线气泡, 适合低延迟高带宽域。

9 组合并行与工程决策

9.1 没有单一最优原语

Recap: LLM parallelism table..

What are each of the parallelism primitives good for?

	Sync overhead	Memory	Bandwidth	Batch size	Easy to use?
DDP/ZeRO1	Per-batch	No scaling	2 * # param	Linear	Very
FSDP (ZeRO3)	3x Per-FSDP block	Linear	3 * # param	Linear	Very
Pipeline	Per-pipeline	Linear	Activations	Linear	NO
Tensor+seq	2x transformer block	Linear	8*activations per-layer all-reduce	No impact	No

Have to balance limited resource – memory, bandwidth, batch size

图 28: DDP/ZeRO、FSDP、流水线和张量 + 序列并行的资源对比。

视频证据时间窗: 01:14:10–01:14:50。

若总设备数分解为数据、张量和流水线三个维度,

$$p = p_{DP} p_{TP} p_{PP}.$$

- p_{TP} : 通常先在单节点高速互连范围内取值;
- p_{PP} : 用于跨节点扩展模型深度和显存;
- p_{DP} : 在模型装下后扩展吞吐。

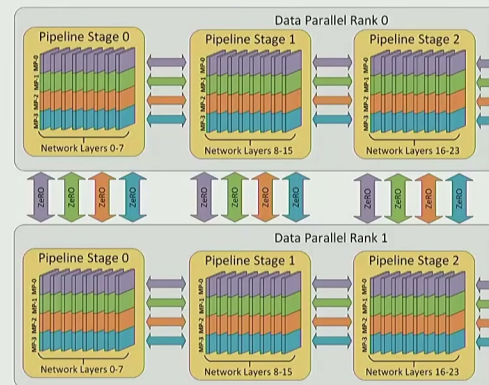
‘3D parallelism’ – putting it all together

Simple rules of thumb from the literature.

1. Until your model fits in memory..
 - Tensor parallel up to GPUs / machine
 - Pipeline parallel across machines
 - › (Or use Zero-3, depending on BW)

2. Then until you run out of GPUs
 - Scale the rest of the way with data parallel

If your batch size is small.. gradient accumulate to trade higher batch sizes for better communication efficiency.



Stanford

图 29: 3D 并行把张量、流水线和数据轴映射到不同设备组。

视频证据时间窗: 01:16:25–01:17:05。

一个简化选择目标是最小化

$$T_{\text{step}} = \max(T_{\text{comp}}, T_{\text{comm}}) + T_{\text{bubble}} + T_{\text{exposed}}$$

- T_{comp} : 每卡有效计算;
- T_{comm} : 可被计算重叠的通信;
- T_{bubble} : 流水线填充、排空和不均衡;
- T_{exposed} : 无法隐藏的 collective、同步与尾部。

同时必须满足显存约束

$$M_{\text{state}} + M_{\text{act}} + M_{\text{temp}} \leq M_{\text{gpu}}$$

- M_{state} : 参数、梯度和优化器;
- M_{act} : 前后向保存的激活;
- M_{temp} : all-gather buffer、workspace 与碎片;
- 应给 OOM 波动和运行时峰值留下余量。

Scaling strategies from Narayanan 2021

Efficient Large-Scale Language Model Training on GPU Clusters
Using Megatron-LM

Deepak Narayanan^{1*}, Mohammad Shoeybi¹, Jared Casper¹, Patrick LeGresley¹,
Mostafa Patwary¹, Vijay Korthukanti¹, Dmitri Vainbrand¹, Predhvi Kashinkunti¹,
Julie Bernauer², Bryan Catanzaro³, Amar Phanishayee³, Matei Zaharia³
¹NVIDIA ²Stanford University ³Microsoft Research

Number of parameters (billion)	Attention heads	Hidden size	Number of layers	Tensor model-parallel size	Pipeline model-parallel size	Number of GPUs	Batch size	Achieved teraFLOP/s per GPU	Percentage of theoretical peak FLOP/s	Achieved aggregate petaFLOP/s	DP size
1.7	24	2304	24	1	1	32	512	137	44%	4.4	32
3.6	32	3072	30	2	1	64	512	138	44%	8.8	32
7.5	32	4096	36	4	1	128	512	142	46%	18.2	32
18.4	48	6144	40	8	1	256	1024	135	43%	34.6	32
39.1	64	8192	48	8	2	512	1536	138	44%	70.8	32
76.1	80	10240	60	8	4	1024	1792	140	45%	143.8	32
145.6	96	12288	80	8	8	1536	2304	148	47%	227.1	24
310.1	128	16384	96	8	16	1920	2160	155	50%	297.4	15
529.6	128	20480	105	8	35	2520	2520	163	52%	410.2	9
1008.0	160	25600	128	8	64	3072	3072	163	52%	502.0	6

Notes

- Tensor parallel first up to 8, then caps out at 8.
- Pipeline parallel goes up to make the model fit.
- Data parallel gradually decreases with scale, with the largest model having DP=6

Stanford

图 30: Dolma、DeepSeek、Yi、Llama 3、Gemma 等公开系统采用不同组合。

视频证据时间窗：01:19:40–01:20:20。

实用选择顺序

先用张量并行或 FSDP 让单个 stage 装下；节点内优先张量 + 序列并行；跨节点用流水线减少高频 collective；模型装下后把剩余设备给数据并行。若批量太小，再回退数据并行度并重新平衡其他轴。

公开配置不能原样复制

不同系统的 GPU、网络、序列长度、精度、MoE、批量和软件栈不同。表格只能提供候选范围；最终配置必须用本地内存峰值、通信带宽、step time 和 loss 曲线验证。

四张必须保留的运行图

每卡显存随时间、每条 GPU stream 的时间线、collective 大小与带宽、各 stage 的计算时间。只有把这四类证据放在一起，才能区分容量不足、通信暴露、流水线气泡和负载不均。

9.2 本章小结

大规模训练通常需要多维并行，而不是寻找唯一“最好”的原语。数据并行负责吞吐，FSDP 负责状态容量，流水线负责深度与慢链路，张量/序列并行负责节点内宽度与激活。正确组合来自显存账本、 α - β 通信模型、批量边界和实测时间线。

10 并行方案设计工作坊

10.1 先建立完整资源账本

为一个待训练模型选并行策略时，先列静态状态：低精度参数、master weight、梯度、两个 Adam moment；再列动态状态：每层保存的残差流、注意力和 MLP 中间量、通信 bucket、临时 all-gather 参数、算子 workspace 与内存碎片。每项标明 dtype、shape、复制因子、分片因子、创建时刻和释放时刻。只有生命周期完整，静态相加才不会低估峰值。

随后建立计算账本：每层前向、反向激活梯度、权重梯度的 FLOPs 与实测时间；建立通信账本：collective 类型、进程组、张量字节、调用频率和链路。最后列优化约束：允许的全局 batch、最小 microbatch、目标序列长度、容忍的梯度累积和 time-to-quality。这个表格是方案比较的共同输入。

显存余量不能设为零

allocator 碎片、通信库内部 buffer、编译器 workspace 和形状波动都会抬高峰值。一个仅在理想测量中刚好装下的方案，长时间训练很容易 OOM。应依据真实峰值保留明确安全余量。

10.2 从硬件拓扑反推并行维度

先识别最快通信域：同一 GPU、同一 NVLink/NVSwitch 节点、同一机架网络和跨机架网络。张量并行每层多次通信，优先放在最快域；序列并行通常与张量并行共享组；流水线只传 stage 边界激活，可跨较慢链路；数据并行或 FSDP 的大 bucket 通信频率较低，但规模大，要利用分层 collective。

逻辑 rank 排列会直接改变物理流量。若相邻流水线 stage 被放到远端，而张量组跨越慢链路，理论相同的 $DP \times TP \times PP$ 配置可能慢数倍。方案文档应保存 rank 坐标、主机名、GPU 编号和每个 process group 成员，不能只写三个并行度数字。

拓扑映射的验证

分别对候选 TP 组、DP 组和跨 stage 链路跑消息尺寸扫描，保存小消息延迟和大消息带宽。再在训练 profiler 中确认实际 collective 使用预期组。配置文件正确不代表运行时 rank 绑定正确。

10.3 选择 FSDP 粒度

FSDP wrap 粒度决定参数物化峰值、collective 数量和可重叠窗口。整模型一个块时，通信次数少但峰值接近完整模型；每个很小子层一个块时，显存低却产生大量小 all-gather，延迟和 Python 调度暴露。常见折中是按 Transformer block 包裹，让当前块计算覆盖下一块预取。

评估时不能只看平均显存和 step time。还要看每次 all-gather 的消息大小、前向和反向是否各自预取、参数释放是否及时、通信流是否被默认流同步、尾部最后一个块的通信是否暴露。若 activation checkpointing 改变计算时间，原来能覆盖的通信也可能重新暴露，因此两项需联合调优。

ZeRO 级别不是越高越好

ZeRO-1/2 在带宽主导时几乎不增加总字节，且实现简单；ZeRO-3 才提供参数容量缩放，却约增加到三份参数级通信。模型若已装下，盲目升到 ZeRO-3 可能用不必要通信换取无用显存。

10.4 流水线调度与 batch

选择流水线 stage 数后，先按实测时间切层，再决定 microbatch 数。microbatch 太少，填充和排空气泡明显；太多则调度开销、激活队列和通信消息数增长。若采用 1F1B，稳定阶段能降低同时存活激活；若使用零气泡变体，必须单独调度激活梯度和权重梯度，并验证优化器更新边界。

流水线并行会把一个优化步拆成复杂状态机。每个 microbatch 必须携带唯一标识，发送和接收顺序一致，反向梯度能回到对应前向，最后才允许参数更新。任何 stage 的异常都要传播到整个作业，不能让其他 rank 永久阻塞在通信调用。

负载均衡按时间而非层数

相同 Transformer block 通常近似等价，但 embedding、词表输出、MoE 层和长序列注意力可能显著更重。应从 profiler 获取各段时间和显存，用分区算法平衡关键路径，并给最慢 stage 留优化余地。

10.5 张量、序列与上下文轴

张量并行把大型矩阵乘分到多卡，可降低单卡参数和部分激活，但每个 block 引入高频 collective。并行度扩大后，每卡矩阵太窄、通信占比上升，吞吐会出现拐点。公开系统常把 GPU 节点大小视为 TP 的自然上限，但真实最优值还取决于隐藏维度、头数和链路。

序列并行把 layer norm、dropout 等点式操作的激活沿序列切分，补足张量并行留下的复制项。更长上下文还可使用 context parallel/ring attention，把注意力所需的键值块沿序列交换。两者名字相近但目的不同：前者主要缩放点式激活，后者让注意力计算跨设备覆盖完整上下文。

MoE 还会增加专家轴

专家并行把专家参数分给不同 rank，token 经过 all-to-all 到对应专家。它与 DP、TP、PP 的通信模式不同，容易形成负载不均和跨节点拥塞。组合时要把专家组、容量因子和路由分布加入同一账本。

10.6 用实验关闭决策

候选配置至少做三组实验：固定模型和 batch 比较每步时间，固定显存上限比较可训练模型或序列长度，固定目标 loss 比较 time-to-quality。每组都记录模型 FLOPs 利用率、每卡峰值显存、collective 带宽、暴露通信、流水线气泡和最慢 rank。只报告 tokens/s 会掩盖批量、序列或样本口径差异。

逐步扩大设备数并画强缩放曲线。效率开始明显下降的位置就是某个并行轴的经济边界；再用时间线判断是 batch 太小、张量切得太窄、跨节点通信还是流水线不平衡。扩展不是一次选出最大规模，而是寻找每一维的拐点。

失败恢复也是并行设计

设备越多，故障概率越高。检查点必须保存分片参数、优化器、数据位置和 RNG 状态，并能在不同 world size 下重分片或明确拒绝。否则一次节点故障会让高吞吐收益被重新训练成本抵消。

10.7 复习检查

应能从零推导：为什么 ring all-reduce 每 rank 约传两份张量；ZeRO-1 为何显存更省而带宽成本接近 DDP；FSDP 为何约三份参数通信；流水线气泡如何随 stage 和 microbatch 变化；张量并行为何应放在高速域；序列并行解决哪部分激活复制；以及 $DP \times TP \times PP$ 如何映射到物理 rank。

10.8 本章小结

并行方案的质量不由某个框架开关决定，而由完整账本、拓扑映射、时间轴重叠、批量约束和数值等价共同决定。工作坊的产物应是一组可验证假设与运行证据，而不是一串并行度参数。

最后还要建立配置审计表：每次运行保存模型版本、并行维度、rank 映射、全局与局部批量、序列长度、精度、检查点策略、通信库环境和 profiler 摘要。性能比较必须一次只改变少数变量，并说明训练质量口径是否相同。若配置在更大规模失效，要把失效点标成显存、带宽、延迟、气泡、批量或数值问题，而不是笼统写“扩展性不好”。这种可检索的失败边界，往往比单次最高吞吐更能指导下一轮集群和模型设计。团队还应保存运行脚本、环境变量、拓扑文件和原始时间线，让一次成功配置能够在新的作业、驱动和节点上复验。只有结论、证据和复现实入口三者齐全，并行经验才真正成为可继承的工程资产。